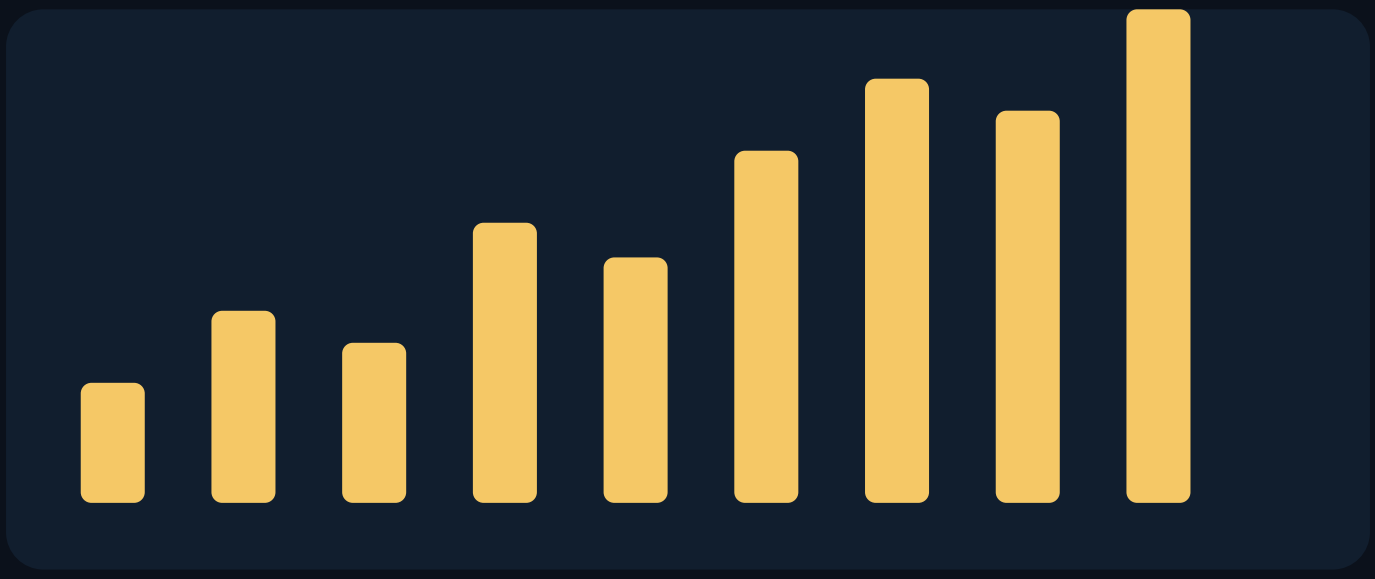
PRACTICAL KNOWLEDGE / DARK EDITION

# Python Automation for Analysts

Build a dependable CSV reporting pipeline with validation, exact money arithmetic, repeatable outputs and useful logs.

By Datta Sable



DATTA SABLE / PROFESSIONAL FIELD GUIDES

THE READER BRIEF

# A guide you can put to work

Automation pays off when the same input leads to the same checked output, and a bad input stops with a clear explanation. A script that saves clicks but silently drops transactions is not a reliable reporting process.

This guide builds a complete CSV-to-summary pipeline without external packages. The sample is small enough to understand by inspection, while the design addresses issues that appear in real reporting work: encodings, duplicate keys, invalid values, missing categories, repeat runs and partial writes.

Use a practice folder and invented records throughout. Read the complete pipeline in chapter 7 before adapting individual fragments. The exercises include deliberate failure cases so you can see whether your process fails clearly rather than merely appearing to run.

## What you need

Python 3.11 or later, a text editor and a local practice folder. The worked pipeline uses only the Python standard library. Commands shown assume Python is available as python; some systems use python3 or an absolute executable path.

## Edition and scope

Original educational examples by Datta Sable. First edition, 28 October 2026. This purchase contains this PDF only. Code, tables and exercises are printed inside; no separate datasets, software licences or editable files are bundled. Product interfaces and availability can change. Check the linked official documentation when applying a platform-specific step.

## Personal learning licence

You may use the techniques in your own work. Do not redistribute, resell or publicly upload the book. The examples use invented data. No customer or employer data is included. Future editions are not included unless the product listing says otherwise.

YOUR LEARNING ROUTE

# Contents

01 / Choose an automation boundary

02 / Create and inspect the sample input

03 / Use Decimal for the monetary contract

04 / Validate every row and preserve exceptions

05 / Separate transformation from file handling

06 / Publish atomically and keep the last good report

07 / Run the complete reporting pipeline

08 / Test the failure cases, not just the happy path

09 / Schedule with visibility and a single owner

10 / Extend the pipeline without losing its contract

Reference desk / Official documentation and next steps

FIELD GUIDE / 01

# Choose an automation boundary

Begin with a process you can describe precisely: take one sales CSV, validate every row, aggregate amount by region and write a summary CSV. Keep source files unchanged. Email delivery, database updates and scheduling are separate responsibilities that can be added after the core transformation is dependable.

## Define success before coding

Input: UTF-8 CSV with OrderID, Region and Amount.
Output: one row per region, sorted by name, plus a log message containing row count and total amount.
Rules: OrderID is unique; Amount is a non-negative decimal with at most two fractional digits; blank Region becomes Unclassified. Any invalid input row stops publication of a new summary.

## A useful folder layout

```
analyst-pipeline/
  input/sales.csv
  output/
  logs/
  report.py
```

Store credentials outside source files if a later version connects to services. The worked version needs no secrets. Run it manually first; scheduling a script before it has useful failure messages only makes faults occur while nobody is watching.

## Practice and answer

Write down what happens when the input file is missing. The preferred outcome is a non-zero exit and a clear message identifying the missing path, while the last successful report remains intact. An empty replacement report would incorrectly look like zero sales.

FIELD GUIDE / 02

# Create and inspect the sample input

Save this exact text as input/sales.csv. Values are invented currency units. Quotes around a field are CSV syntax, not part of its value. Use a real CSV parser rather than splitting each line on commas.

```
OrderID,Region,Amount
A001,West,198.00
A002,South,149.00
A003,West,297.00
A004,North,499.00
A005,,298.00
```

There are five records and total amount 1,441.00. West contributes 495.00, South 149.00, North 499.00 and Unclassified 298.00. Those values are the acceptance baseline for the pipeline.

## Why CSV needs a parser

A quoted value can contain a comma, line break or quote. csv.DictReader handles those rules and gives each row named fields. Open CSV files with newline='' so the csv module controls newline handling. utf-8-sig accepts a common UTF-8 byte-order mark while also reading ordinary UTF-8 input.

## Inspect structure first

Compare the header sequence with the agreed schema. An extra unnamed column can signal a trailing delimiter or malformed export. Missing Amount should fail immediately rather than cause an obscure error halfway through the file.

## Exercise and answer

Change Region on A001 to "West, Retail" and include the surrounding quotes. A proper parser reads one region field. A naive split(',') produces an extra column. Restore the original before comparing the baseline totals.

FIELD GUIDE / 03

# Use Decimal for the monetary contract

Binary floating point is excellent for many numerical tasks, but decimal currency values are easier to reason about with Decimal. Construct Decimal directly from the input string; converting a float first preserves the float approximation instead of the original decimal text.

```
from decimal import Decimal, InvalidOperation

def parse_amount(raw, row_number):
    try:
        value = Decimal(raw.strip())
    except (InvalidOperation, AttributeError):
        raise ValueError(f"row {row_number}: invalid Amount")
    if not value.is_finite() or value < 0:
        raise ValueError(f"row {row_number}: invalid Amount")
    if value != value.quantize(Decimal("0.01")):
        raise ValueError(f"row {row_number}: too many decimals")
    return value
```

This is a focused fragment; chapter 7 includes it in the complete program. The finite check rejects NaN and Infinity, both of which Decimal can represent but a sales report should not accept. The precision rule rejects 12.345 rather than silently choosing a rounding policy.

## Parsing is a business decision

The contract accepts plain decimal strings such as 149.00. It does not accept currency symbols or locale-specific thousands separators. If a source sends 1,234.50, handle that format explicitly before parsing and test it. Blindly removing punctuation can turn 12,50 into 1,250 instead of 12.50.

## Exercise and answer

Try -1, NaN and 0.001. All must fail. Try 0 and 12.30. Both pass. Keep those cases in the regression tests.

FIELD GUIDE / 04

# Validate every row and preserve exceptions

Validation should happen before output publication. For a small file, you can accumulate the summary in memory while validating and only write after the loop succeeds. Large workflows may stage accepted and rejected rows separately, but must still reconcile their counts to the input.

## Identity checks

Trim OrderID and reject empty values. Keep a set of IDs already seen; reject a duplicate instead of guessing which row is newer. If the business supports corrected versions, it needs an explicit version or timestamp rule that this simple pipeline does not have.

## Classification checks

A blank Region becomes Unclassified. This preserves the money and makes the missing attribute visible. A misspelling such as Wset is different: an allowed-value list or reference mapping is needed if the business requires standard region names. Do not silently fuzzy-match reporting dimensions without review.

## Row numbers matter

Start data-row numbering at 2 because the header occupies line 1. Include the row number and field name in a validation error, but avoid logging whole source rows if they may contain confidential values. A good message tells the operator where to look without spreading the data.

## Exercise and answer

Append A001 again with amount 10.00. The run should fail, and a previous successful summary should remain unchanged. A total of 1,451.00 would mean the duplicate escaped validation. Remove the duplicate only after confirming that the program rejects it.

FIELD GUIDE / 05

# Separate transformation from file handling

Pure transformations are easier to test than code that mixes file access, business rules and email. The complete program uses summarise(path) to read and validate, and publish(path, totals) to write. The aggregation returns exact Decimal values rather than preformatted strings.

```
from collections import defaultdict
from decimal import Decimal

totals = defaultdict(lambda: Decimal("0.00"))
totals["West"] += Decimal("198.00")
totals["West"] += Decimal("297.00")
assert totals["West"] == Decimal("495.00")
```

## Deterministic ordering

Sort region names before writing. Otherwise output order can depend on which region appeared first in the source file. Stable ordering makes review and comparison easier and prevents meaningless diffs when the same input rows arrive in a different order.

## Avoid hidden dependencies

Pass the input and output paths to functions rather than relying on the current working directory inside every helper. A scheduled task may start in a different folder from an interactive terminal. Resolve or document paths at the command boundary.

## Exercise and answer

Reverse the order of the five CSV records. The summary values and output row order should remain identical. The row-specific error messages may change if a validation defect moves, but a valid report should not change because of source ordering.

FIELD GUIDE / 06

# Publish atomically and keep the last good report

Writing directly over a report risks leaving a partial file if the process stops midway. Write a temporary file in the destination directory, finish and close it, then replace the destination. Keeping the temporary file on the same filesystem supports an atomic replacement on typical local filesystems; remote storage can have different guarantees.

```
import os, tempfile
from pathlib import Path

destination = Path("output/summary.csv")
destination.parent.mkdir(parents=True, exist_ok=True)
with tempfile.NamedTemporaryFile(
    mode="w", dir=destination.parent, delete=False,
    encoding="utf-8", newline="", suffix=".tmp"
) as handle:
    temporary = Path(handle.name)
    handle.write("Region,Amount\nWest,495.00\n")
os.replace(temporary, destination)
```

This fragment illustrates publication; the next chapter adds CSV formatting and cleanup if writing fails. Atomic replacement prevents a reader from seeing a half-written report, but does not solve overlapping runs or guarantee disaster recovery. Keep backups or versioned outputs when the operational requirement demands them.

## Spreadsheet-safe text

CSV text beginning with =, +, - or @ can be interpreted as a formula by spreadsheet software. Treat externally supplied labels as data, and neutralise formula-leading text in exports intended for Excel. This does not replace validation or a clear file-opening policy.

## Exercise and answer

Make the output folder read-only in a disposable practice copy or use an invalid output location. The process must return failure and retain the previous report. Restore permissions before the next run.

FIELD GUIDE / 07

# Run the complete reporting pipeline

Save this as report.py. It is the full program, not a fragment. Keep the sample input beside it in the documented folder structure.

```
import csv, logging, os, sys, tempfile
from collections import defaultdict
from decimal import Decimal, InvalidOperation
from pathlib import Path

def summarise(source):
    totals = defaultdict(lambda: Decimal("0.00"))
    seen = set()
    with source.open(encoding="utf-8-sig", newline="") as f:
        reader = csv.DictReader(f)
        if reader.fieldnames != ["OrderID", "Region", "Amount"]:
            raise ValueError("unexpected CSV headers")
        for number, row in enumerate(reader, 2):
            if None in row or any(v is None for v in row.values()):
                raise ValueError(f"row {number}: malformed CSV")
            key = row["OrderID"].strip()
            if not key or key in seen:
                raise ValueError(f"row {number}: duplicate/empty ID")
            seen.add(key)
            try:
                value = Decimal(row["Amount"].strip())
                valid = value.is_finite() and value >= 0
                valid = valid and value == value.quantize(Decimal(".01"))
            except InvalidOperation:
                valid = False
            if not valid:
                raise ValueError(f"row {number}: invalid Amount")
            totals[row["Region"].strip() or "Unclassified"] += value
    if not seen:
        raise ValueError("no data rows")
    return dict(totals), len(seen)

def safe_text(value):
    return "'" + value if value[:1] in "=+-@" else value

def publish(destination, totals):
    destination.parent.mkdir(parents=True, exist_ok=True)
    temporary = None
    try:
        with tempfile.NamedTemporaryFile(
            mode="w", dir=destination.parent, delete=False,
            encoding="utf-8", newline="", suffix=".tmp"
        ) as f:
            temporary = Path(f.name)
            writer = csv.writer(f)
```

```
            writer.writerow(["Region", "Amount"])
            for region in sorted(totals):
                writer.writerow([safe_text(region), f"{totals[region]:.2f}"])
        os.replace(temporary, destination)
    finally:
        if temporary and temporary.exists():
            temporary.unlink()

def main():
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    if len(sys.argv) != 3:
        logging.error("usage: python report.py input.csv output.csv")
        return 2
    source, destination = map(Path, sys.argv[1:])
    if source.resolve() == destination.resolve():
        logging.error("input and output must differ")
        return 2
    try:
        totals, count = summarise(source)
        publish(destination, totals)
        logging.info("rows=%s total=%s", count, sum(totals.values()))
        return 0
    except (OSError, ValueError):
        logging.exception("report failed; output was not published")
        return 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Run: python report.py input/sales.csv output/summary.csv. The log must report five rows and total 1441.00. The program does not send data to an external service.

FIELD GUIDE / 08

# Test the failure cases, not just the happy path

A test suite should challenge the contract. It should not merely repeat the same implementation line in a second file. Start with a baseline that a person can independently calculate, then alter one thing at a time.

## Acceptance cases

Valid sample: five rows, four regions, total 1,441.00.
Duplicate ID: non-zero exit; previous output unchanged.
Negative, NaN or three-decimal amount: non-zero exit.
Missing header or extra field: non-zero exit.
Blank region: output includes Unclassified, 298.00.
Shuffled valid input: identical summary.
Same input/output path: usage failure without source modification.

## A focused unit test

```
import tempfile
from decimal import Decimal
from pathlib import Path
from report import summarise

with tempfile.TemporaryDirectory() as folder:
    source = Path(folder) / "sales.csv"
    source.write_text(
        "OrderID,Region,Amount\nA1,West,0.10\nA2,West,0.20\n",
        encoding="utf-8"
    )
    totals, count = summarise(source)
    assert totals["West"] == Decimal("0.30")
    assert count == 2
```

## Test publication separately

Seed the output with a known marker, run invalid input and compare the bytes afterward. This proves the last good report was preserved. For a valid run, parse the output with csv.DictReader instead of relying on a visually plausible file. Keep a small regression folder with both valid and invalid input examples.

FIELD GUIDE / 09

# Schedule with visibility and a single owner

Only schedule the script after manual runs and failure tests pass. Use absolute paths for the interpreter, script, input and output. A scheduled task's working directory and environment may differ from your terminal session.

## Define operating rules

Choose the expected source arrival time and reporting cutoff. Specify what happens if the file is late: fail and alert, reuse a clearly dated prior report, or wait within a bounded window. Never label yesterday's output as today's without an explicit status.

Prevent overlapping runs through the scheduler or a suitable lock. Atomic file replacement protects readers from partial output, but two concurrent jobs can still race and publish in the wrong order. If multiple source batches are valid, include a batch identifier in output and logs.

## Log for diagnosis

Capture start time, source identifier, accepted row count, total amount, duration and final status. Do not write credentials or complete source rows to logs. Keep log retention aligned with the team's data policy. A success message should be emitted only after publication completes.

## Exercise and answer

Rename the input before a scheduled practice run. Confirm the task returns a failure code and the failure is visible in the scheduler history or chosen log destination. Restore the source and run again. The recovery should require no code edit or manual manipulation of the summary file.

FIELD GUIDE / 10

# Extend the pipeline without losing its contract

New requirements should enter through a controlled boundary. Adding multiple CSV files means defining duplicate-batch behaviour. Adding email means recording whether the mail provider accepted a message separately from whether a recipient received it. Adding an API means handling timeouts, pagination and rate limits.

## Three safe extensions

Batch provenance: add SourceFile and BatchDate to staged data, then reconcile file-level row counts before aggregation.
Configuration: accept an agreed region mapping file and validate that each source key maps once. Preserve an exceptions report for missing keys.
Versioned output: write summary-YYYY-MM-DD.csv and update a separate latest pointer only after validation and publication.

## Know when memory is the limit

The seen-ID set grows with the number of rows. For files too large for the available memory, move duplicate checks and aggregation into a suitable database or disk-backed workflow. Do not remove uniqueness checking merely to make the script finish.

## Final handoff

Deliver report.py, sample input, expected summary, failure cases and the runbook. State that amounts are non-negative in this exercise; real returns require a defined signed-amount or separate-return model. Record the Python version tested and the scheduler owner. Good automation makes the next operator confident about both success and failure.

KEEP LEARNING

# Reference desk

Official references checked while preparing this edition. Use these pages for syntax, feature availability and the latest product details. The exercises and business scenarios in this book are original.

### Python: CSV reading and writing

https://docs.python.org/3/library/csv.html

### Python: decimal arithmetic

https://docs.python.org/3/library/decimal.html

### Python: pathlib filesystem paths

https://docs.python.org/3/library/pathlib.html

### Python: logging

https://docs.python.org/3/library/logging.html

### Python: os.replace

https://docs.python.org/3/library/os.html#os.replace

## Make the next step observable

Run the supplied sample, then prove the duplicate-ID and bad-amount failures preserve your last successful report. Once those checks are repeatable, adapt one input contract at a time and retain the same acceptance evidence.

Questions about your purchased edition? Visit dattasable.com/contact and include your order number. Independently published; no endorsement by the software vendors named in this guide is implied.